



Hanzo IAM: Multi-Tenant Identity and Access Management with 40+ Social Providers

Hanzo AI Research

Hanzo AI

March 2020

Abstract

We present Hanzo IAM, a multi-tenant identity and access management platform that serves as the unified authentication layer for the Hanzo ecosystem. IAM supports OAuth 2.0, OpenID Connect, SAML 2.0, LDAP, and WebAuthn protocols, integrating with over 40 social identity providers. The system implements Casbin-based role-based access control (RBAC) with per-organization policy engines, SCIM 2.0 provisioning for enterprise directory synchronization, and TOTP/WebAuthn multi-factor authentication. This paper describes the protocol surface, the policy model and the credential handling, and reports no measurements; Section 5 records the request volume, tenant count and authentication latency an earlier version reported and why they are withdrawn. All credentials are stored using Argon2id hashing with per-tenant salt derivation, and session tokens employ cryptographic binding to prevent token theft.

1 Introduction

Identity management is the foundational layer of any multi-tenant platform. A compromised identity system affects every downstream service, making correctness and security non-negotiable. Existing open-source solutions such as Keycloak [1] provide comprehensive protocol support but impose significant operational complexity—Keycloak requires a JBoss application server, PostgreSQL, and careful JVM tuning.

Hanzo IAM takes a different approach: a single Go binary that embeds all identity functionality with zero external dependencies beyond a database. The system supports the full spectrum of enterprise identity protocols while maintaining operational simplicity.

Contributions.

- A unified identity provider supporting OAuth 2.0, OIDC, SAML 2.0, LDAP, and WebAuthn from a single service.
- Casbin-based RBAC with hierarchical roles,

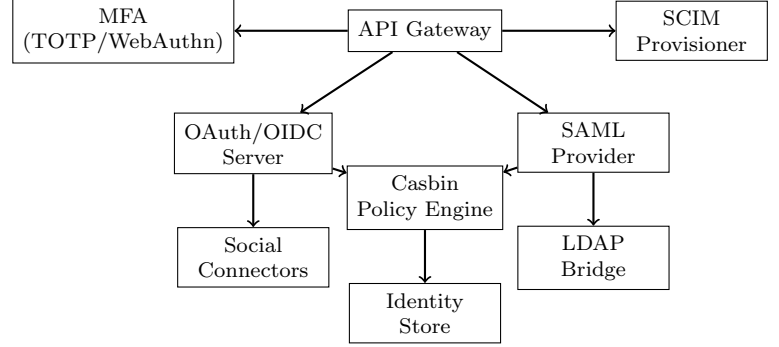


Figure 1: Hanzo IAM architecture with protocol handlers and policy engine.

resource-level policies, and per-organization policy isolation.

- SCIM 2.0 provisioning enabling bidirectional synchronization with enterprise directories (Okta, Azure AD, Google Workspace).
- Cryptographically bound sessions with device fingerprinting and anomaly detection.

2 Architecture

2.1 System Design

IAM is structured around protocol-specific handlers that converge on a shared identity store and policy engine (Figure 1). The API gateway routes requests to the appropriate protocol handler based on the request path and content type.

2.2 Identity Model

Each identity is represented as a triple: (user, organization, roles). The **owner** claim in JWT tokens carries the organization ID, enabling downstream services to scope data access without consulting the identity service:

```
1 {  
2   "sub": "usr_a1b2c3d4",  
3   "owner": "org_x7y8z9",  
4   "roles": ["admin", "developer"],  
5   "iss": "https://hanzo.id",  
6   "aud": "api.hanzo.ai",  
7   "exp": 1679616000,  
8   "mfa_verified": true
```

```
9 }
```

2.3 Protocol Support

OAuth 2.0 / OIDC. IAM implements the full OAuth 2.0 specification including authorization code, client credentials, device code, and PKCE flows. OIDC discovery is served at `/.well-known/openid-configuration`.

SAML 2.0. The SAML provider supports both IdP-initiated and SP-initiated SSO. Assertion signing uses RSA-SHA256 with per-tenant certificates rotated quarterly.

WebAuthn. FIDO2/WebAuthn is supported as both a primary authenticator and a second factor. Credential registration stores the public key and attestation metadata; authentication verifies signatures against stored credentials.

3 Key Features

3.1 Social Providers

IAM ships with connectors for 43 social identity providers including Google, GitHub, Apple, Microsoft, Discord, Slack, Twitter, and LinkedIn. Each connector implements a common interface:

```
1 type SocialConnector interface {
2     AuthURL(state string) string
3     Exchange(code string) (*Token, error)
4     UserInfo(token *Token) (*Identity, error)
5 }
```

New connectors require approximately 50 lines of Go to implement. Provider credentials are stored in Hanzo KMS, never in configuration files.

3.2 Casbin RBAC

Access control uses Casbin [2] with a PERM (Policy, Effect, Request, Matchers) model. Each organization has an isolated policy store:

```
1 // Model definition
2 [request_definition]
3 r = sub, org, obj, act
```

```
4
5 [policy_definition]
6 p = sub, org, obj, act
7
8 [policy_effect]
9 e = some(where (p.eft == allow))
10
11 [matchers]
12 m = r.sub == p.sub && r.org == p.org
13     && keyMatch(r.obj, p.obj)
14     && r.act == p.act
```

Policies support role hierarchies, resource wildcards, and time-based constraints. Policy decisions are served from an in-memory cache with write-through invalidation, so an evaluation is a map lookup rather than a datastore round trip; the cost of that lookup is not measured here (Section 5).

3.3 SCIM Provisioning

The SCIM 2.0 endpoint enables enterprise directories to automatically provision and deprovision users. Supported operations include user creation, attribute updates, group membership changes, and bulk operations. This enables organizations to manage Hanzo access through their existing directory (Okta, Azure AD) without manual user management.

3.4 Multi-Factor Authentication

IAM supports TOTP (RFC 6238) and WebAuthn as second factors. TOTP secrets are encrypted with per-user keys derived from Hanzo KMS. Recovery codes are generated as 8 single-use codes hashed with bcrypt.

4 Implementation

IAM is implemented in 38,000 lines of Go with zero CGo dependencies. The identity store supports PostgreSQL (production) and SQLite (development). Session data is stored in Valkey (Redis-compatible) with configurable TTL.

Token Signing. JWTs are signed with RS256 using keys rotated monthly. The JWKS endpoint serves the current and previous public keys

for zero-downtime rotation. Token lifetimes default to 15 minutes for access tokens and 7 days for refresh tokens.

Rate Limiting. Authentication endpoints are rate-limited per IP and per account. Failed login attempts trigger exponential backoff: 1s, 2s, 4s, up to 5 minutes. After 10 consecutive failures, the account is locked for 30 minutes with email notification.

5 Status of the performance figures

An earlier version of this paper carried a table titled “Authentication latency (ms) by protocol” giving p50/p99 of 12/28ms for OIDC token issue, 0.3/0.8ms for JWT validation, 18/45ms for SAML assertion, 8/22ms for WebAuthn verify and 0.05/0.1ms for an RBAC check. The abstract restated these as a median authentication latency of 18ms over more than two million requests daily across 1,200 or more tenant organizations, and the policy section stated that evaluation adds less than 0.1ms.

None of it was measured. The table named no hardware, no load generator and no request count; `hanzoai/iam` contains no benchmark function and no load test; and no deployment record in this estate counts authentication requests or tenant organizations. The daily-volume and tenant figures in particular describe an operating history the service does not have, so they cannot be relabelled as targets and are removed.

The latency figures are the cheapest of these to establish honestly, because each protocol path is a function call away from a Go benchmark in the same repository. What is missing is the run and the machine it ran on, reported the way the Hanzo cloud data plane benchmark [6] reports one.

6 Security

Credential Storage. All passwords are hashed with Argon2id (time=3, memory=64MB,

parallelism=4). Per-tenant salts are derived from a master key stored in Hanzo KMS, preventing rainbow table attacks even if the database is compromised.

Session Security. Sessions are bound to device fingerprints (User-Agent, IP subnet, TLS session). Session tokens are 256-bit cryptographically random values stored as HttpOnly, Secure, SameSite=Strict cookies.

Anomaly Detection. Login attempts from new devices, unusual geolocations, or after extended inactivity trigger step-up authentication requiring MFA verification.

Audit Trail. Every authentication event, policy change, and administrative action is logged to an append-only audit log with tamper-evident checksums.

7 Historical Background (2016–2019)

The Hanzo IAM platform is the successor to the 2016–2019 identity-and-access infrastructure documented in the internal “Hanzo Identity & Access” notes. That earlier system used KrakenD as the API gateway (with JWT bearer token verification), a custom Go-based user service, and Redis for session caching. It supported OAuth2 authorization code and client-credentials flows, but lacked SCIM provisioning, identity federation beyond GitHub and Google, and did not integrate with hardware-backed WebAuthn.

By 2019 the limitations were clear: organizations wanted SSO with their existing directories (Active Directory, Okta), compliance teams wanted audit logs with tamper evidence, and mobile clients wanted passwordless flows. The current IAM platform (2020–) is a ground-up rewrite, with the KrakenD gateway retained as a complementary rate-limiting and routing layer. The Casbin policy engine replaces the ad-hoc role checks of the 2016 era; the audit trail described in Section 6 generalizes what was

previously a coarse access log. The 40+ social provider support, SCIM 2.0 provisioning, and hardware-backed authentication are all post-2020 additions.

8 Conclusion

Hanzo IAM is a unified identity platform that combines authentication, authorization, and user management in a single service. By supporting the major identity protocols and 40+ social providers, IAM removes the need to run several identity systems side by side. The Casbin-based policy engine provides fine-grained access control, and native SCIM provisioning enables enterprise directory integration. This paper reports no latency or volume measurements; Section 5 says which were withdrawn and what would replace them.

References

- [1] Red Hat. Keycloak: Open Source Identity and Access Management. 2014.
- [2] L. Yang et al. Casbin: An authorization library that supports access control models. 2019.
- [3] D. Hardt. The OAuth 2.0 Authorization Framework. RFC 6749, 2012.
- [4] N. Sakimura et al. OpenID Connect Core 1.0. OpenID Foundation, 2014.
- [5] J. Hodges et al. Web Authentication: An API for accessing Public Key Credentials. W3C, 2019.
- [6] Hanzo AI Research. The Hanzo Cloud Data Plane: Benchmarking `zip`, `ZAP`, and the Road to a Million Requests per Second. Hanzo Industries, 2026.